
Platforma E-learningowa

System Zarządzania Kursami Online

Dokumentacja Techniczna Projektu

Bazy Danych – PostgreSQL 16

Zespół projektowy:

Antoni Pokrzywa Bartosz Łyczak
Bartosz Śnieg Łukasz Bilski

Czerwiec 2026

Spis treści

1	Wprowadzenie i analiza wymagań	3
1.1	Cel projektu	3
1.2	Zakres funkcjonalny	3
1.3	Skład zespołu i podział odpowiedzialności	3
2	Projekt logiczny – diagram ERD	4
2.1	Opis encji i relacji	4
2.2	Relacje między tabelami	4
3	Projekt fizyczny – schemat bazy danych	5
3.1	Tabela <code>users</code>	5
3.2	Tabela <code>user_profiles</code>	6
3.3	Tabela <code>courses</code>	6
3.4	Tabela <code>course_enrollments</code>	6
3.5	Tabela <code>lessons</code>	7
3.6	Tabela <code>tests</code>	7
3.7	Tabela <code>grades</code>	7
4	Normalizacja	7
4.1	Pierwsza postać normalna (1NF)	7
4.2	Druga postać normalna (2NF)	8
4.3	Trzecia postać normalna (3NF)	8
5	Implementacja – kod SQL	8
5.1	Tworzenie tabel	8
6	Wyzwalacze (Triggers)	10
6.1	Wyzwalacz sprawdzający zapis na kurs	10
6.2	Wyzwalacz blokujący duplikaty ocen	11
6.3	Wyzwalacz logujący nową ocenę	11
7	Widoki	12
7.1	Widok <code>v_ranking_studentow</code>	12
7.2	Widok <code>v_statystyki_kursu</code>	13
7.3	Widok <code>v_postepy_studenta</code>	13
8	Transakcje i poziomy izolacji	14
8.1	Zastosowanie transakcji	14
8.1.1	Scenariusz 1 – Rejestracja użytkownika z profilem	14
8.1.2	Scenariusz 2 – Zapis na kurs (REPEATABLE READ)	14
8.1.3	Scenariusz 3 – Wystawienie oceny (SERIALIZABLE)	15
8.2	Poziomy izolacji	15
9	Bezpieczeństwo danych – role i uprawnienia	15
9.1	Struktura ról w systemie PostgreSQL	15
9.2	Przyznane uprawnienia	16
9.3	Zasada najmniejszych uprawnień	16

10 Zaawansowane zapytania SQL	17
10.1 Zagnieżdżone podzapytanie – studenci bez ocen	17
10.2 Statystyki per kurs z wielokrotnym JOIN	17
10.3 Funkcje okienkowe – ranking z RANK()	18
10.4 Filtrowanie z HAVING	18
11 Dane testowe i studium przypadków	18
11.1 Dane testowe	18
11.2 Studium przypadku 1: Rola elearning_readonly	19
11.3 Studium przypadku 2: Rola elearning_teacher	19
12 Podsumowanie	19

1 Wprowadzenie i analiza wymagań

1.1 Cel projektu

Celem projektu jest zaprojektowanie i implementacja relacyjnej bazy danych dla platformy e-learningowej (systemu zarządzania kursami online) w środowisku PostgreSQL 16. System umożliwia zarządzanie użytkownikami z podziałem na role (student, nauczyciel, administrator), tworzenie i prowadzenie kursów złożonych z lekcji, przeprowadzanie testów wiedzy oraz rejestrowanie i analizowanie wyników studentów.

1.2 Zakres funkcjonalny

System realizuje następujące funkcje:

1. **Zarządzanie użytkownikami** – rejestracja, przechowywanie danych uwierzytliwiających (email, hash hasła) i przypisanie roli.
2. **Profile użytkowników** – dane personalne (imię, nazwisko, bio, avatar) w oddzielnej tabeli w relacji 1:1 z kontem.
3. **Zarządzanie kursami** – tworzenie kursów przez nauczycieli, opis i śledzenie daty utworzenia.
4. **Zapisy na kursy** – realizacja relacji wiele-do-wielu (M:N) między studentami a kursami z kontrolą unikalności zapisu.
5. **Lekcje** – uporządkowana treść kursu z możliwością określenia kolejności wyświetlania.
6. **Testy wiedzy** – sprawdziany powiązane z lekcjami z określoną maksymalną liczbą punktów.
7. **Oceny** – rejestracja wyników studentów z dokładnością do 0.01 punktu (typ NUMERIC).
8. **Bezpieczeństwo** – wyzwalacze chroniące integralność danych, podział ról i uprawnień w bazie danych.

1.3 Skład zespołu i podział odpowiedzialności

Projekt został zrealizowany w zespole czteroosobowym. Każdy członek zespołu był liderem wyznaczonego obszaru prac:

- **Antoni Pokrzywa (apokrzywa)** – Koordynator projektu (Zarabia 1.6 mln zł rocznie), odpowiedzialny za analizę wymagań, przygotowanie schematu konceptualnego, normalizację bazy do 3NF oraz fizyczną implementację struktury tabel, kluczy i sekwencji.
- **Bartosz Łyczak (blyczak)** – Analityk danych, odpowiedzialny za projektowanie logicznych relacji, implementację zaawansowanych widoków systemowych oraz optymalizację zapytań agregujących.

- **Bartosz Śnieg (bsnieg)** – Programista bazy danych, odpowiedzialny za automatyzację logiki biznesowej, implementację funkcji proceduralnych w języku PL/pgSQL oraz wdrożenie wyzwalaczy.
- **Łukasz Bilski (lbilski)** – Administrator systemu, odpowiedzialny za warstwę bezpieczeństwa (zarządzanie rolami i uprawnieniami), kontrolę transakcji oraz konfigurację poziomów izolacji.

2 Projekt logiczny – diagram ERD

2.1 Opis encji i relacji

Schemat bazy danych składa się z **siedmiu tabel** powiązanych relacjami:

Tabela 1: Zestawienie encji

Encja	Klucz główny	Opis
users	id	Dane uwierzytelniające
user_profiles	id	Profile personalne (1:1)
courses	id	Kursy online
course_enrollments	id	Zapisy na kursy (M:N)
lessons	id	Lekcje w kursach
tests	id	Testy wiedzy
grades	id	Oceny i wyniki

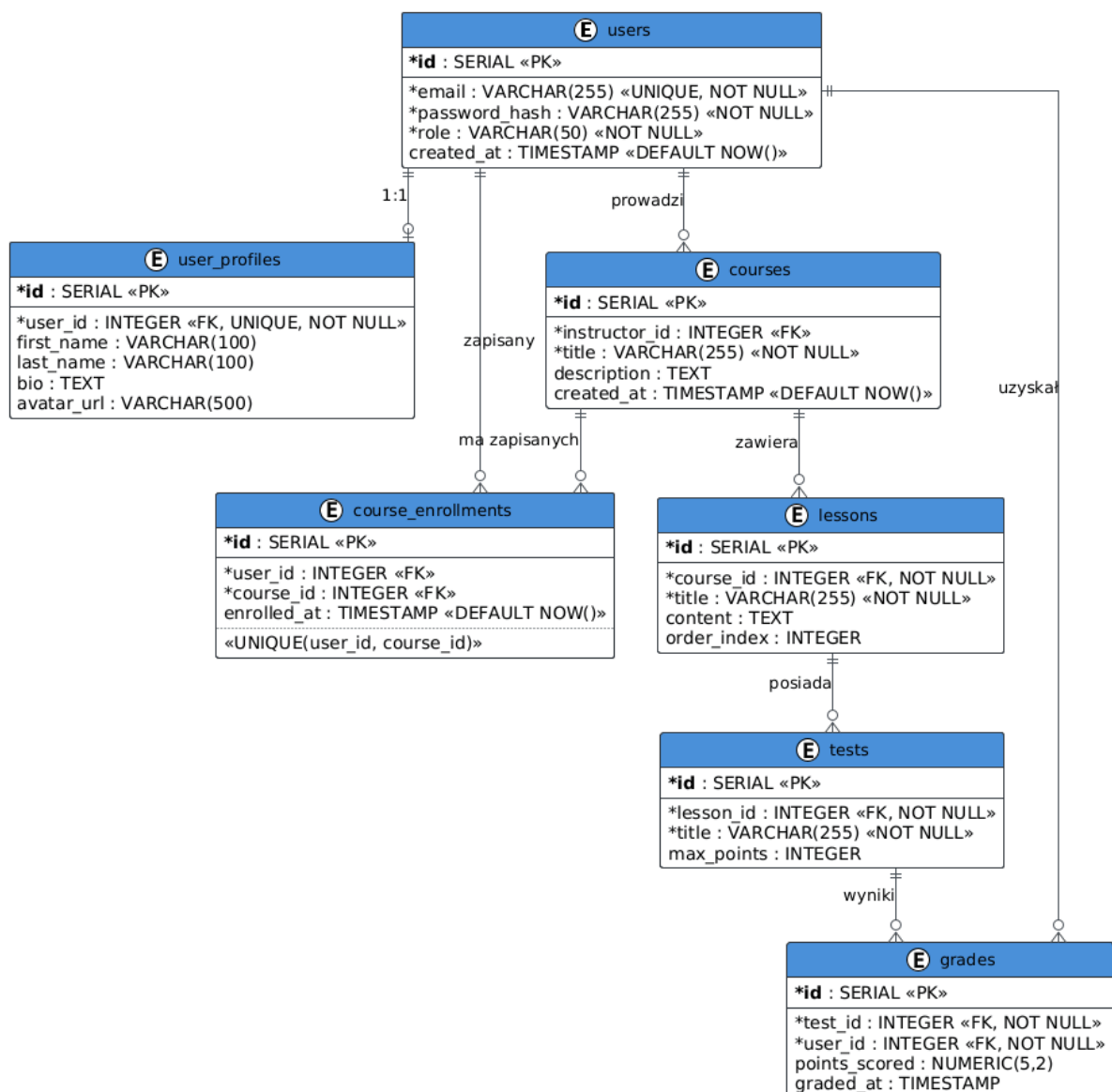
2.2 Relacje między tabelami

Tabela 2: Klucze obce i relacje

Tabela podrzędna	Klucz obcy	Tabela nadrzędna	ON DELETE
user_profiles	user_id	users	CASCADE
courses	instructor_id	users	SET NULL
course_enrollments	user_id	users	CASCADE
course_enrollments	course_id	courses	CASCADE
lessons	course_id	courses	CASCADE
tests	lesson_id	lessons	CASCADE
grades	test_id	tests	CASCADE
grades	user_id	users	CASCADE

Diagram ERD

Diagram związków encji (ERD) został wygenerowany za pomocą narzędzia PlantUML i załączony jako osobny plik graficzny. Przedstawia wszystkie siedem tabel z zaznaczonymi kluczami głównymi, obcymi oraz typami relacji (1:1, 1:N, M:N).



Rysunek 1: Diagram związków encji (ERD) dla platformy e-learningowej

3 Projekt fizyczny – schemat bazy danych

3.1 Tabela users

Tabela 3: Struktura tabeli users

Kolumna	Typ	Ograniczenia	Opis
id	SERIAL	PK, AUTO	Identyfikator
email	VARCHAR(255)	NOT NULL, UNIQUE	Adres e-mail
password_hash	VARCHAR(255)	NOT NULL	Hash hasła
role	VARCHAR(50)	NOT NULL, CHECK	Rola systemowa
created_at	TIMESTAMP	DEFAULT NOW()	Data rejestracji

3.2 Tabela user_profiles

Tabela 4: Struktura tabeli user_profiles

Kolumna	Typ	Ograniczenia	Opis
id	SERIAL	PK, AUTO	Identyfikator
user_id	INTEGER	FK, UNIQUE, NOT NULL	Właściciel profilu
first_name	VARCHAR(100)	–	Imię
last_name	VARCHAR(100)	–	Nazwisko
bio	TEXT	–	Biografia
avatar_url	VARCHAR(500)	–	URL awatara

3.3 Tabela courses

Tabela 5: Struktura tabeli courses

Kolumna	Typ	Ograniczenia	Opis
id	SERIAL	PK, AUTO	Identyfikator
instructor_id	INTEGER	FK, SET NULL	Prowadzący
title	VARCHAR(255)	NOT NULL	Tytuł kursu
description	TEXT	–	Opis kursu
created_at	TIMESTAMP	DEFAULT NOW()	Data utworzenia

3.4 Tabela course_enrollments

Tabela 6: Struktura tabeli course_enrollments

Kolumna	Typ	Ograniczenia	Opis
id	SERIAL	PK, AUTO	Identyfikator
user_id	INTEGER	FK, CASCADE	Student
course_id	INTEGER	FK, CASCADE	Kurs
enrolled_at	TIMESTAMP	DEFAULT NOW()	Data zapisu

Uwaga strukturalna

Na parę kolumn (user_id, course_id) nałożono unikalny indeks (UNIQUE), aby student nie mógł zapisać się na ten sam kurs dwukrotnie.

3.5 Tabela lessons

Tabela 7: Struktura tabeli lessons

Kolumna	Typ	Ograniczenia	Opis
id	SERIAL	PK, AUTO	Identyfikator
course_id	INTEGER	FK, NOT NULL	Kurs nadrzędny
title	VARCHAR(255)	NOT NULL	Tytuł lekcji
content	TEXT	–	Treść lekcji
order_index	INTEGER	–	Kolejność

3.6 Tabela tests

Tabela 8: Struktura tabeli tests

Kolumna	Typ	Ograniczenia	Opis
id	SERIAL	PK, AUTO	Identyfikator
lesson_id	INTEGER	FK, NOT NULL	Lekcja nadrzędna
title	VARCHAR(255)	NOT NULL	Nazwa testu
max_points	INTEGER	–	Max punktów

3.7 Tabela grades

Tabela 9: Struktura tabeli grades

Kolumna	Typ	Ograniczenia	Opis
id	SERIAL	PK, AUTO	Identyfikator
test_id	INTEGER	FK, NOT NULL	Test
user_id	INTEGER	FK, NOT NULL	Student
points_scored	NUMERIC(5,2)	–	Zdobyte punkty
graded_at	TIMESTAMP	DEFAULT NOW()	Data oceny

Zgodność z 3NF

Tabela **grades** nie przechowuje informacji, czy student „zdał”, ponieważ zależy to od wartości **max_points** w tabeli **tests** – przechowywanie takiej informacji stworzyłoby zależność przechodnią, naruszając trzecią postać normalną.

4 Normalizacja

4.1 Pierwsza postać normalna (1NF)

Wszystkie atrybuty we wszystkich tabelach przechowują wartości atomowe (brak kolumn wielowartościowych, brak powtarzających się grup). Każda tabela posiada jednoznaczny

klucz główny (SERIAL).

4.2 Druga postać normalna (2NF)

W żadnej tabeli nie występuje złożony klucz główny – każda tabela ma klucz syntetyczny (SERIAL), więc nie może wystąpić częściowa zależność funkcyjna. Tabela `course_enrollments` realizuje relację M:N z własnym kluczem głównym i ograniczeniem `UNIQUE` na parze (`user_id`, `course_id`).

4.3 Trzecia postać normalna (3NF)

Zbadano zależności przechodnie:

- `users` ↔ `user_profiles`: dane personalne wydzielono do osobnej tabeli – brak zależności przechodniej w tabeli `users`.
- `courses`: `instructor_id` → dane nauczyciela znajdują się w tabeli `users`, nie w `courses`.
- `grades`: nie przechowuje `max_points` ani statusu „zdał/nie zdał” – te informacje wynikają z tabeli `tests`.
- `lessons`: nie przechowuje tytułu kursu – zależy od `course_id` → `courses.title`.

Żadna kolumna niebędąca kluczem nie zależy od innej kolumny niebędącej kluczem. Schemat spełnia wymagania 3NF.

5 Implementacja – kod SQL

5.1 Tworzenie tabel

```
1 CREATE TABLE public.users (  
2     id          SERIAL          NOT NULL,  
3     email       VARCHAR(255)    NOT NULL,  
4     password_hash VARCHAR(255)  NOT NULL,  
5     role        VARCHAR(50)     NOT NULL,  
6     created_at  TIMESTAMP WITHOUT TIME ZONE  
7                 DEFAULT CURRENT_TIMESTAMP,  
8     CONSTRAINT users_pkey PRIMARY KEY (id),  
9     CONSTRAINT users_email_key UNIQUE (email),  
10    CONSTRAINT users_role_check  
11        CHECK (role IN ('student', 'teacher', 'admin'))  
12 );
```

Listing 1: Definicja tabeli users

```
1 CREATE TABLE public.user_profiles (  
2     id          SERIAL          NOT NULL,  
3     user_id     INTEGER          NOT NULL,  
4     first_name  VARCHAR(100),
```

```
5 last_name VARCHAR(100),
6 bio TEXT,
7 avatar_url VARCHAR(500),
8 CONSTRAINT user_profiles_pkey PRIMARY KEY (id),
9 CONSTRAINT user_profiles_user_id_key UNIQUE (user_id),
10 CONSTRAINT user_profiles_user_id_fkey
11 FOREIGN KEY (user_id)
12 REFERENCES public.users(id) ON DELETE CASCADE
13 );
```

Listing 2: Definicja tabeli user_profiles

```
1 CREATE TABLE public.courses (
2 id SERIAL NOT NULL,
3 instructor_id INTEGER,
4 title VARCHAR(255) NOT NULL,
5 description TEXT,
6 created_at TIMESTAMP WITHOUT TIME ZONE
7 DEFAULT CURRENT_TIMESTAMP,
8 CONSTRAINT courses_pkey PRIMARY KEY (id),
9 CONSTRAINT courses_instructor_id_fkey
10 FOREIGN KEY (instructor_id)
11 REFERENCES public.users(id) ON DELETE SET NULL
12 );
```

Listing 3: Definicja tabeli courses

```
1 CREATE TABLE public.course_enrollments (
2 id SERIAL NOT NULL,
3 user_id INTEGER,
4 course_id INTEGER,
5 enrolled_at TIMESTAMP WITHOUT TIME ZONE
6 DEFAULT CURRENT_TIMESTAMP,
7 CONSTRAINT course_enrollments_pkey PRIMARY KEY (id),
8 CONSTRAINT course_enrollments_user_course_key
9 UNIQUE (user_id, course_id),
10 CONSTRAINT course_enrollments_user_id_fkey
11 FOREIGN KEY (user_id)
12 REFERENCES public.users(id) ON DELETE CASCADE,
13 CONSTRAINT course_enrollments_course_id_fkey
14 FOREIGN KEY (course_id)
15 REFERENCES public.courses(id) ON DELETE CASCADE
16 );
```

Listing 4: Definicja tabeli course_enrollments

```
1 CREATE TABLE public.grades (
2 id SERIAL NOT NULL,
3 test_id INTEGER NOT NULL,
4 user_id INTEGER NOT NULL,
5 points_scored NUMERIC(5,2),
6 graded_at TIMESTAMP WITHOUT TIME ZONE
7 DEFAULT CURRENT_TIMESTAMP,
```

```
8      CONSTRAINT grades_pkey PRIMARY KEY (id),
9      CONSTRAINT grades_test_id_fkey
10         FOREIGN KEY (test_id)
11         REFERENCES public.tests(id) ON DELETE CASCADE,
12      CONSTRAINT grades_user_id_fkey
13         FOREIGN KEY (user_id)
14         REFERENCES public.users(id) ON DELETE CASCADE
15 );
```

Listing 5: Definicja tabeli grades

6 Wyzwalacze (Triggers)

6.1 Wyzwalacz sprawdzający zapis na kurs

Przed każdym wstawieniem rekordu do tabeli `grades` wywoływana jest funkcja `sprawdz_zapis_na_kurs`. Sprawdza ona, czy student jest zapisany na kurs, do którego należy dany test. Zapobiega to wystawianiu ocen studentom, którzy nie uczestniczą w kursie.

```
1 CREATE FUNCTION public.sprawdz_zapis_na_kurs()
2     RETURNS trigger LANGUAGE plpgsql AS
3 $$
4 DECLARE
5     v_course_id INTEGER;
6     v_enrolled  BOOLEAN;
7 BEGIN
8     SELECT l.course_id INTO v_course_id
9         FROM public.tests t
10        JOIN public.lessons l ON t.lesson_id = l.id
11        WHERE t.id = NEW.test_id;
12
13     SELECT EXISTS (
14         SELECT 1 FROM public.course_enrollments
15         WHERE user_id = NEW.user_id
16               AND course_id = v_course_id
17     ) INTO v_enrolled;
18
19     IF NOT v_enrolled THEN
20         RAISE EXCEPTION
21             'Student nie jest zapisany na kurs.';
22     END IF;
23
24     RETURN NEW;
25 END;
26 $$;
```

Listing 6: Funkcja wyzwalacza `sprawdz_zapis_na_kurs()`

```
1 CREATE TRIGGER trg_sprawdz_zapis
2     BEFORE INSERT ON public.grades
3     FOR EACH ROW
```

```
EXECUTE FUNCTION public.sprawdz_zapis_na_kurs();
```

Listing 7: Rejestracja wyzwalacza zapisu

6.2 Wyzwalacz blokujący duplikaty ocen

Ze względów integralności danych system blokuje możliwość wystawienia wielokrotnej oceny temu samemu studentowi z tego samego testu.

```
1 CREATE FUNCTION public.blokuj_duplikat_oceny()  
2 RETURNS trigger LANGUAGE plpgsql AS  
3 $$  
4 BEGIN  
5     IF EXISTS (  
6         SELECT 1 FROM public.grades  
7         WHERE test_id = NEW.test_id  
8         AND user_id = NEW.user_id  
9     ) THEN  
10        RAISE EXCEPTION  
11        'Student posiada juz ocene z tego testu.';  
12    END IF;  
13  
14    RETURN NEW;  
15 END;  
16 $$;
```

Listing 8: Funkcja wyzwalacza blokuj_duplikat_oceny()

```
1 CREATE TRIGGER trg_blokuj_duplikat_oceny  
2 BEFORE INSERT ON public.grades  
3 FOR EACH ROW  
4 EXECUTE FUNCTION public.blokuj_duplikat_oceny();
```

Listing 9: Rejestracja wyzwalacza blokady duplikatów

6.3 Wyzwalacz logujący nową ocenę

Po wystawieniu oceny system automatycznie loguje informację o kursie, teście, studencie i wyniku procentowym. Informacja jest widoczna w logach PostgreSQL (poziom NOTICE).

```
1 CREATE FUNCTION public.loguj_nowa_ocene()  
2 RETURNS trigger LANGUAGE plpgsql AS  
3 $$  
4 DECLARE  
5     v_course_title VARCHAR(255);  
6     v_test_title   VARCHAR(255);  
7     v_max_points   INTEGER;  
8     v_procent      NUMERIC(5,2);  
9 BEGIN  
10    SELECT c.title, t.title, t.max_points  
11    INTO v_course_title, v_test_title, v_max_points  
12    FROM public.tests t
```

```

13      JOIN public.lessons l ON t.lesson_id = l.id
14      JOIN public.courses c ON l.course_id = c.id
15      WHERE t.id = NEW.test_id;
16
17      IF v_max_points > 0 THEN
18          v_procent := ROUND(
19              (NEW.points_scored / v_max_points) * 100, 2);
20      ELSE
21          v_procent := 0;
22      END IF;
23
24      RAISE NOTICE '[OCENA] Kurs: "%" | Test: "%" | Wynik: %%',
25          v_course_title, v_test_title, v_procent;
26
27      RETURN NEW;
28  END;
29  $$;

```

Listing 10: Funkcja wyzwalacza loguj_nowa_ocene()

```

1 CREATE TRIGGER trg_loguj_ocene
2     AFTER INSERT ON public.grades
3     FOR EACH ROW
4     EXECUTE FUNCTION public.loguj_nowa_ocene();

```

Listing 11: Rejestracja wyzwalacza logowania

7 Widoki

7.1 Widok v_ranking_studentow

Zwraca ranking studentów z łącznymi punktami, maksymalnymi możliwymi punktami i procentem poprawnych odpowiedzi. Łączy dane z czterech tabel.

```

1 CREATE VIEW public.v_ranking_studentow AS
2 SELECT
3     u.id AS student_id,
4     up.first_name || ' ' || up.last_name
5     AS imie_nazwisko,
6     u.email,
7     COUNT(g.id) AS liczba_testow,
8     ROUND(SUM(g.points_scored), 2) AS suma_punktow,
9     ROUND(SUM(t.max_points), 2) AS max_punktow,
10    CASE
11        WHEN SUM(t.max_points) > 0
12        THEN ROUND(
13            (SUM(g.points_scored) / SUM(t.max_points))
14            * 100, 2)
15        ELSE 0
16    END AS procent_poprawnych
17 FROM users u

```

```

18 JOIN user_profiles up ON u.id = up.user_id
19 JOIN grades g         ON u.id = g.user_id
20 JOIN tests t          ON g.test_id = t.id
21 WHERE u.role = 'student'
22 GROUP BY u.id, up.first_name, up.last_name, u.email
23 ORDER BY procent_poprawnych DESC;

```

Listing 12: Widok rankingu studentów

7.2 Widok v_statystyki_kursu

Agreguje statystyki kursów: liczbę lekcji, testów, zapisanych studentów oraz średni wynik procentowy.

```

1 CREATE VIEW public.v_statystyki_kursu AS
2 SELECT
3     c.id AS kurs_id,
4     c.title AS tytul_kursu,
5     ui.first_name || ' ' || ui.last_name AS instruktor,
6     COUNT(DISTINCT l.id) AS liczba_lekcji,
7     COUNT(DISTINCT t.id) AS liczba_testow,
8     COUNT(DISTINCT ce.user_id) AS liczba_studentow,
9     COALESCE(ROUND(AVG(g.points_scored), 2), 0)
10     AS srednia_punktow
11 FROM courses c
12 LEFT JOIN users u         ON c.instructor_id = u.id
13 LEFT JOIN user_profiles ui ON u.id = ui.user_id
14 LEFT JOIN lessons l       ON c.id = l.course_id
15 LEFT JOIN tests t         ON l.id = t.lesson_id
16 LEFT JOIN course_enrollments ce ON c.id = ce.course_id
17 LEFT JOIN grades g        ON t.id = g.test_id
18 GROUP BY c.id, c.title, ui.first_name, ui.last_name
19 ORDER BY liczba_studentow DESC;

```

Listing 13: Widok statystyk kursów

7.3 Widok v_postepy_studenta

Pokazuje postępy każdego studenta w poszczególnych kursach: ile testów rozwiązał, jaki procent ukończenia i jaką średnią uzyskał.

```

1 CREATE VIEW public.v_postepy_studenta AS
2 SELECT
3     u.id AS student_id,
4     up.first_name || ' ' || up.last_name
5     AS imie_nazwisko,
6     c.title AS kurs,
7     COUNT(DISTINCT t.id) AS testy_w_kursie,
8     COUNT(DISTINCT g.test_id) AS testy_rozwiazane,
9     CASE
10         WHEN COUNT(DISTINCT t.id) > 0
11         THEN ROUND(

```

```

12         (COUNT(DISTINCT g.test_id)::NUMERIC
13          / COUNT(DISTINCT t.id)) * 100, 1)
14     ELSE 0
15 END AS procent_ukonczenia,
16 COALESCE(ROUND(AVG(g.points_scored), 2), 0)
17 AS srednia_punktow
18 FROM users u
19 JOIN user_profiles up      ON u.id = up.user_id
20 JOIN course_enrollments ce ON u.id = ce.user_id
21 JOIN courses c            ON ce.course_id = c.id
22 LEFT JOIN lessons l       ON c.id = l.course_id
23 LEFT JOIN tests t         ON l.id = t.lesson_id
24 LEFT JOIN grades g        ON t.id = g.test_id
25                          AND g.user_id = u.id
26 WHERE u.role = 'student'
27 GROUP BY u.id, up.first_name, up.last_name, c.title
28 ORDER BY u.id, c.title;

```

Listing 14: Widok postępów studenta

8 Transakcje i poziomy izolacji

8.1 Zastosowanie transakcji

System korzysta z transakcji PostgreSQL w scenariuszach wymagających atomowości.

8.1.1 Scenariusz 1 – Rejestracja użytkownika z profilem

```

1 BEGIN;
2
3 INSERT INTO users (email, password_hash, role)
4 VALUES ('nowy.student@mail.pl',
5         '$2b$12$EXAMPLE_HASH', 'student');
6
7 INSERT INTO user_profiles
8     (user_id, first_name, last_name, bio)
9 VALUES (
10     currval('users_id_seq'),
11     'Nowy', 'Student',
12     'Wlasnie dolaczyl do platformy!'
13 );
14
15 COMMIT;

```

Listing 15: Transakcja rejestracji użytkownika

8.1.2 Scenariusz 2 – Zapis na kurs (REPEATABLE READ)

```

1 BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
2
3 INSERT INTO course_enrollments (user_id, course_id)
4 SELECT currval('users_id_seq'), 1
5 WHERE NOT EXISTS (
6     SELECT 1 FROM course_enrollments
7     WHERE user_id = currval('users_id_seq')
8         AND course_id = 1
9 );
10
11 COMMIT;

```

Listing 16: Transakcja zapisu na kurs

8.1.3 Scenariusz 3 – Wystawienie oceny (SERIALIZABLE)

```

1 BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
2
3 INSERT INTO grades (test_id, user_id, points_scored)
4 VALUES (1, currval('users_id_seq'), 16.50);
5
6 COMMIT;

```

Listing 17: Transakcja wystawienia oceny z poziomem SERIALIZABLE

8.2 Poziomy izolacji

Tabela 10: Poziomy izolacji transakcji zastosowane w systemie

Poziom izolacji	Zastosowanie	Uzasadnienie
READ COMMITTED (domyślny)	Odczyty widoków, raporty	Wydajność, brak brudnych odczytów
REPEATABLE READ	Zapis na kurs, raporty	Spójny obraz danych w trakcie operacji
SERIALIZABLE	Wystawianie ocen	Pełna atomowość, brak anomalii

9 Bezpieczeństwo danych – role i uprawnienia

9.1 Struktura ról w systemie PostgreSQL

W celu odzwierciedlenia struktury zespołu oraz wdrożenia mechanizmów bezpieczeństwa, w systemie PostgreSQL zaimplementowano cztery konta użytkowników o zróżnicowanych poziomach uprawnień:

- apokrzywa – Główny właściciel bazy danych, posiadający pełne prawa DDL i DML.
- blyczak – Rola dedykowana analitykowi danych, z uprawnieniami do odczytu i modyfikacji danych oraz pełnym dostępem do widoków.
- bsnieg – Konto deweloperskie z uprawnieniami proceduralnymi, umożliwiające tworzenie i modyfikację funkcji oraz wyzwalaczy (*TRIGGER*).
- lbilski – Rola administracyjna dedykowana do zarządzania uprawnieniami, kontroli transakcji oraz nadzoru nad sekwencjami.

9.2 Przyznane uprawnienia

```
1  -- blyczak (analityk danych)
2  GRANT ALL ON TABLE public.users TO blyczak;
3  GRANT ALL ON TABLE public.user_profiles TO blyczak;
4  GRANT ALL ON TABLE public.courses TO blyczak;
5  GRANT ALL ON TABLE public.course_enrollments TO blyczak;
6  GRANT ALL ON TABLE public.lessons TO blyczak;
7  GRANT ALL ON TABLE public.tests TO blyczak;
8  GRANT ALL ON TABLE public.grades TO blyczak;
9
10 -- bsnieg (programista bazy danych)
11 GRANT SELECT, INSERT, UPDATE, TRIGGER
12     ON ALL TABLES IN SCHEMA public TO bsnieg;
13 GRANT ALL ON ALL FUNCTIONS
14     IN SCHEMA public TO bsnieg;
15
16 -- lbilski (administrator)
17 GRANT SELECT, INSERT, UPDATE, DELETE
18     ON ALL TABLES IN SCHEMA public TO lbilski;
19 GRANT SELECT
20     ON ALL SEQUENCES IN SCHEMA public TO lbilski;
```

Listing 18: Skrypt przyznawania uprawnień

9.3 Zasada najmniejszych uprawnień

W środowisku produkcyjnym zastosowano bardziej restrykcyjny podział uprawnień:

```
1  -- Rola tylko do odczytu (monitoring, audyt)
2  CREATE ROLE elearning_readonly;
3  GRANT SELECT ON ALL TABLES
4      IN SCHEMA public TO elearning_readonly;
5  GRANT SELECT ON ALL SEQUENCES
6      IN SCHEMA public TO elearning_readonly;
7
8  -- Rola nauczyciela (ocenianie)
9  CREATE ROLE elearning_teacher;
10 GRANT SELECT, INSERT, UPDATE
11     ON TABLE public.grades TO elearning_teacher;
```

```
12 GRANT SELECT ON TABLE public.users
13     TO elearning_teacher;
14 GRANT SELECT ON TABLE public.tests
15     TO elearning_teacher;
16 GRANT SELECT ON TABLE public.courses
17     TO elearning_teacher;
```

Listing 19: Konfiguracja produkcyjna

10 Zaawansowane zapytania SQL

10.1 Zagnieżdżone podzapytanie – studenci bez ocen

```
1 SELECT u.email, up.first_name, up.last_name
2 FROM users u
3 JOIN user_profiles up ON u.id = up.user_id
4 WHERE u.role = 'student'
5     AND u.id NOT IN (
6         SELECT DISTINCT g.user_id
7         FROM grades g
8     );
```

Listing 20: Studenci, którzy nie ukończyli żadnego testu

10.2 Statystyki per kurs z wielokrotnym JOIN

```
1 SELECT
2     up.first_name || ' ' || up.last_name AS student,
3     c.title AS kurs,
4     COUNT(g.id) AS liczba_testow,
5     ROUND(SUM(g.points_scored), 2) AS suma_punktow,
6     CASE
7         WHEN SUM(t.max_points) > 0
8         THEN ROUND(
9             (SUM(g.points_scored) / SUM(t.max_points))
10            * 100, 1)
11         ELSE 0
12     END AS procent
13 FROM users u
14 JOIN user_profiles up ON u.id = up.user_id
15 JOIN grades g         ON u.id = g.user_id
16 JOIN tests t          ON g.test_id = t.id
17 JOIN lessons l        ON t.lesson_id = l.id
18 JOIN courses c        ON l.course_id = c.id
19 GROUP BY up.first_name, up.last_name, c.title
20 ORDER BY procent DESC;
```

Listing 21: Wyniki studentów pogrupowane per kurs

10.3 Funkcje okienkowe – ranking z RANK()

```
1 SELECT
2     c.title AS kurs,
3     up.first_name || ' ' || up.last_name AS student,
4     ROUND(SUM(g.points_scored), 2) AS suma_punktow,
5     RANK() OVER (
6         PARTITION BY c.id
7         ORDER BY SUM(g.points_scored) DESC
8     ) AS pozycja_w_kursie
9 FROM grades g
10 JOIN tests t      ON g.test_id = t.id
11 JOIN lessons l    ON t.lesson_id = l.id
12 JOIN courses c    ON l.course_id = c.id
13 JOIN users u      ON g.user_id = u.id
14 JOIN user_profiles up ON u.id = up.user_id
15 GROUP BY c.id, c.title, up.first_name, up.last_name
16 ORDER BY c.title, pozycja_w_kursie;
```

Listing 22: Ranking studentów w kursie z RANK()

10.4 Filtrowanie z HAVING

```
1 SELECT
2     c.title,
3     COUNT(DISTINCT g.user_id) AS liczba_studentow,
4     ROUND(AVG(
5         (g.points_scored / t.max_points) * 100
6     ), 2) AS sredni_procent
7 FROM courses c
8 JOIN lessons l ON c.id = l.course_id
9 JOIN tests t   ON l.id = t.lesson_id
10 JOIN grades g  ON t.id = g.test_id
11 WHERE t.max_points > 0
12 GROUP BY c.id, c.title
13 HAVING AVG(
14     (g.points_scored / t.max_points) * 100) > 70
15 ORDER BY sredni_procent DESC;
```

Listing 23: Kursy ze średnią powyżej 70%

11 Dane testowe i studium przypadków

11.1 Dane testowe

Baza zawiera predefiniowane dane testowe:

- **8 użytkowników** – 1 administrator (admin@elearning.pl), 2 nauczycieli (Jan Kowalski, Anna Nowak) i 5 studentów.

- **8 profili** – powiązanych relacją 1:1 z kontami użytkowników.
- **3 kursy** – „Podstawy SQL”, „Zaawansowany Python”, „Wprowadzenie do ML”; prowadzone przez dwóch nauczycieli.
- **10 zapisów** na kursy – studenci zapisani na 2–3 kursy.
- **12 lekcji** – po 4 lekcje na kurs.
- **9 testów** – po 3 testy na kurs, z punktacją 15–50 pkt.
- **20 ocen** – realistyczne wyniki studentów, od 9 do 47.5 punktów.

11.2 Studium przypadku 1: Rola `elearning_readonly`

Rola `elearning_readonly` przeznaczona jest do audytu i monitoringu. Może wykonywać zapytania `SELECT` na wszystkich tabelach i widokach, ale system restrykcyjnie blokuje próby modyfikacji danych (`INSERT`, `UPDATE`, `DELETE`).

11.3 Studium przypadku 2: Rola `elearning_teacher`

Nauczyciel może odczytywać dane kursów, testów i studentów oraz wystawiać i modyfikować oceny. Nie posiada jednak uprawnień do tworzenia ani usuwania kursów, lekcji czy testów (wyłącznie administrator).

12 Podsumowanie

W ramach projektu zaprojektowano i zaimplementowano kompletny system bazodanowy dedykowany do zarządzania platformą e-learningową w środowisku PostgreSQL 16. Wszystkie założone cele inżynierskie oraz wymagania funkcjonalne zostały w pełni osiągnięte:

- **Pełna normalizacja (3NF)** – struktura wszystkich siedmiu tabel została zoptymalizowana, eliminując redundancję danych oraz niepożądane zależności przechodnie.
- **Integralność referencyjna** – powiązania relacyjne zabezpieczono kluczami obcymi z precyzyjnie dobranymi regułami kaskadowymi (`ON DELETE CASCADE`, `SET NULL`). Poprawność ról wymuszono więzami `CHECK`.
- **Proceduralna automatyzacja** – zaimplementowano trzy wyzwalacze (triggery) z funkcjami PL/pgSQL: weryfikacja zapisu na kurs, blokada duplikatów ocen, logowanie wyników.
- **Zaawansowana analityka** – utworzono trzy widoki relacyjne (`v_ranking_studentow`, `v_statystyki_kursu`, `v_postepy_studenta`), dynamicznie agregujące dane.
- **Bezpieczeństwo transakcyjne** – operacje krytyczne ustrukturyzowano w bloki transakcyjne z jawnym przypisaniem poziomów izolacji (`READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE`).
- **Strukturyzacja ról i uprawnień** – wdrożono kompletny model bezpieczeństwa odzwierciedlający podział odpowiedzialności w zespole (apokrzywa, blyczak, bsnieg, lbilski) oraz role produkcyjne (`elearning_readonly`, `elearning_teacher`).

- **Realistyczne środowisko testowe** – baza zasilona spójnym zestawem danych: 8 użytkowników, 3 kursy, 12 lekcji, 9 testów, 20 ocen.